

Table of Contents

Claude Code Orchestration — Install Guide

Version 3.0.0 · Updated 2026-05-23

This guide installs a private Claude Code orchestration architecture: an engineering orchestrator + 23 specialist subagents + 36 skills + 5 guardrail hooks, designed to **compose with** the official Anthropic plugin stack (15 plugins) rather than duplicate them.

What's new in v3:

- meta-architecture-reviewer agent — audits the plugin itself for overlap, drift, and unclear handoffs
- Single source of truth for the recommended-plugins list — reference/recommended-plugins.json (no more drift across CLAUDE.md / plugin.json / INSTALL / bootstrap)
- Graceful degradation — each agent now documents what it falls back to when an official plugin isn't installed
- Consolidated review pipeline — ONE policy reviewer + ONE bug-finder pass per unit, no more 5-reviewer pile-up
- Limitations.md + EdgeCases.md as hard gates before Plan-Final.md
- Pre-implementation response must include Edge Cases + Limitations sections
- Postman MCP section + workbook path decision documented

The install is split into three parts: install 15 official plugins, install this plugin, run the repo bootstrap.

1. What this is

A policy + glue layer on top of the official Claude Code plugin ecosystem. The official plugins do the heavy lifting (TDD enforcement, code-review, browser automation, doc lookups, GitHub API). This plugin enforces *your* engineering policy on top of them: pre-implementation discipline, plan review cycles, architecture enforcement, no-assumptions rule, official-docs-first, assumption-validation tests with pre/post comparison, comment policy (no tickets / branches / plans in code), PR merge-conflict awareness, local git hygiene.

Critical principle: official plugins are tools that the custom agents use. They never replace the custom agents. Every custom agent enforces a custom output contract regardless of which official plugin it leverages.

2. Prerequisites

- **Claude Code** 2.1.143 or later. Check: `claude --version`. Upgrade: `npm i -g @anthropic-ai/claude-code@latest` or `brew upgrade claude-code`.
 - **Git** 2.30+.
 - **bash** (macOS, Linux, WSL).
 - A GitHub account (or any Git host) if you want to publish your own marketplace.
-

3. Install — Path A: from a Git marketplace (recommended)

3.1 Install the recommended official plugins

The canonical list of recommended plugins is in **marketplace/plugins/claude-code-orchestration/reference/recommended-plugins.json** — this is the SINGLE SOURCE OF TRUTH. Don't paste a list from somewhere else; ask the bootstrap to print it for you:

```
bash /path/to/bootstrap/install_repo_bootstrap.sh --print-plugins
```

That command reads `reference/recommended-plugins.json` and prints the exact `/plugin` install lines to paste into Claude Code. Plugins marked optional only matter if your work touches the relevant stack (Microsoft = `microsoft-docs`, UI = `frontend-design`, etc.).

The 15 plugins, summarized:

Plugin

context7
code-review
feature-dev
superpowers
github
atlassian
playwright
chrome-devtools-mcp
serena
claude-md-management
commit-commands
claude-code-setup
pr-review-toolkit
microsoft-docs
frontend-design

Each plugin has a `fallback_when_missing` field in `reference/recommended-plugins.json` documenting what happens if it's not installed.

3.2 Publish this marketplace once

```
cd marketplace
git init && git add . && git commit -m "cc-orchestration marketplace v2"
git remote add origin git@github.com:<you>/cc-orchestration.git
git push -u origin main
```

3.3 Install this plugin

Inside Claude Code:

```
/plugin marketplace add <you>/cc-orchestration
/plugin install claude-code-orchestration@cc-orchestration
/reload-plugins
```

3.4 Bootstrap each repo

```
cd /path/to/your/repo
bash /path/to/bootstrap/install_repo_bootstrap.sh --repo .
```

3.5 Verify

```
/plugin      # all 16 plugins Installed, no Errors
/agents      # engineering-orchestrator + 21 specialists
/hooks       # 5 hooks from claude-code-orchestration
/mcp         # MCPs from the official plugins
/memory      # CLAUDE.md from this plugin loaded
```

4. Install — Path B: local plugin (no Git push)

```
git clone <this-package> ~/cc-orchestration
# Then inside Claude Code:
/plugin marketplace add ~/cc-orchestration/marketplace
/plugin install claude-code-orchestration@cc-orchestration
/reload-plugins
bash ~/cc-orchestration/bootstrap/install_repo_bootstrap.sh --repo /path/to/repo
```

5. The 23 agents (v3)

Pre-existing custom agents (19, from v1):

- engineering-orchestrator — coordinates the others
- requirements-product-analyst — no-assumptions, asks questions, produces acceptance criteria
- external-documentation-researcher — uses the right doc source per topic (microsoft-docs / context7 / etc.)
- codebase-researcher — uses serena / codegraphcontext / etc.

- implementation-planner — produces staged plans
- coding-agent — implements one approved unit
- code-reviewer — reviews every file against the stage + custom policy; combines with /code-review
- test-agent — runs validation matrix; uses superpowers TDD when available
- documentation-reviewer — keeps docs/ignored/** clean
- performance-reviewer — anti-overengineering
- refactor-cleanup-agent — minimal cleanup
- git-version-control-agent — read-only git
- merge-conflict-resolver — approved-files-only resolver
- pr-creator — uses repo conventions + github plugin + commit-commands
- pr-reviewer — uses pr-review-toolkit + /code-review
- backend-bug-finder — finds bugs, doesn't fix
- frontend-bug-finder — uses playwright + chrome-devtools-mcp
- pre-push-guardian — strict final gate
- worktree-manager — branch + worktree mechanics

NEW in v2 (3 agents + 1 coordinator):

- architecture-enforcer — verifies code aligns with codebase architecture
- data-architect — DB schema, migrations, indexes, data contracts
- comment-policy-checker — blocks ticket/branch/plan refs + AI attribution from code & commits
- parallel-research-coordinator — orchestrates parallel researchers + consistency review + gap-fill rounds

NEW in v3:

- meta-architecture-reviewer — audits the cc-orchestration plugin itself for overlap, drift, unclear handoffs. Read-only. Produces Plugin-Self-Review-`<ts>.md`. Run this periodically (e.g., after any $v(n+1)$ of the plugin) to catch the kind of issues this audit found.
-

6. The 36 skills (v2)

Pre-existing (27, from v1): implementation-feedback-loop, implementation-planning, codebase-contextualization, validation-matrix, workbook-management, documentation-refresh, safe-git-commit, merge-conflict-handling, pr-convention-detection, branch-creation, worktree-handoff, agent-team-decision, root-cause-convergence, backend-error-triage, observability-correlation, database-debugging, backend-security-scan, frontend-error-triage, browser-reproduction-debugging, frontend-observability-correlation, design-system-visual-regression, accessibility-performance-triage, run-documentation-review, run-pre-push-guardian, run-pr-review, run-backend-bug-finder, run-frontend-bug-finder.

NEW in v2 (9 skills):

- greenfield-vs-brownfield — pick the mode + apply the corresponding stricter rules
 - plan-review-cycle — multi-round plan convergence with 3 parallel reviewers
 - parallel-codebase-research-cycle — multi-round parallel research + consistency reviewer + gap-fill
 - parallelization-decision — forces explicit human approval before any fan-out
 - plan-folder-organization — one folder per feature, slug never contains ticket ID
 - pr-merge-conflict-wait — polls GH mergeability + required checks after PR opens
 - assumption-validation-tests — pre/post-test comparison artifacts (complements superpowers)
 - official-docs-first — routing rule (vendor-specific source first, context7 generic fallback)
 - gitignore-local-hygiene — .git/info/exclude patterns to stop personal files following branches
-

7. Workflow at a glance

orchestrator



requirements-product-analyst (uses /brainstorming from superpowers)



external-documentation-researcher (microsoft-docs / context7 / ...)



codebase-researcher → parallel-research-coordinator (if multi-area)
uses serena + codegraphcontext + codeql + codanna + tree-sitter



architecture-enforcer (uses feature-dev's code-architect as input)



data-architect (if DB schema/migrations involved)



implementation-planner → plan-review-cycle (3 reviewers × N rounds)



HUMAN APPROVAL of Plan-Final.md + parallelization-decision



per stage:

coding-agent (+ superpowers TDD red-green-refactor)

codebase-contextualization

architecture-enforcer

code-reviewer (+ /code-review from code-review plugin)

test-agent (+ assumption-validation-tests pre/post)

documentation-reviewer

comment-policy-checker



pr-creator (github + commit-commands plugins, repo conventions)



pr-merge-conflict-wait (polls mergeable + required checks)

8. Mandatory rules (from CLAUDE.md)

1. **Verify before claiming** — no fact stated without verification.
 2. **No assumptions** — explicit pre-implementation response with Open Questions.
 3. **Official docs first** — vendor-specific source > context7 > WebSearch. Cite.
 4. **Codebase research first** — use serena + the right MCP for the task.
 5. **Implementation feedback loop** — code → context → architecture → review → test → docs → comment-policy.
 6. **TDD validate assumptions** — test fails before, passes after. Compare.
 7. **Parallelization opt-in only** — ask the human before fan-out.
 8. **Greenfield vs. brownfield** — brownfield default; drift requires plan declaration + human approval.
 9. **Plan review cycle** — Plan-v1 → 3 parallel reviewers → consolidate → revise → re-review; only Plan-Final.md is implementable.
 10. **No process metadata in code or commits** — tickets, branches, plans, stages, phases, AI attribution all blocked.
 11. **Local git hygiene** — .git/info/exclude for personal patterns; never .gitignore.
 12. **Database safety** — explicit confirmation for any write/migration/admin.
 13. **PR merge-conflict wait** — poll GH after PR opens.
-

9. Daily commands

Start a normal session (orchestrator auto-active)

claude

Override active agent

claude --agent data-architect

Branch / worktree

~/local/share/cc-orchestration/new-branch.sh feature/something

~/local/share/cc-orchestration/new-worktree.sh feature/something ../wt-something

Print the plugin install commands

~/local/share/cc-orchestration/install_repo_bootstrap.sh --print-plugins

10. The strengthened comment + commit policy

This was the user's #1 pain point. v2 enforces it at FOUR layers:

Layer

post-edit-policy-check.sh hook (PostToolUse Edit/Write)

comment-policy-checker agent

pre-push-guardian agent

Git pre-commit + commit-msg + pre-push hooks (installed by bootstrap)

To bypass legitimately (e.g., a string literal that happens to match), set `ALLOW_COMMENT_POLICY_BYPASS=1` for that commit only.

11. The local-git hygiene story

(From the user's requirements doc, verbatim.)

When you work on a branch in git, untracked files are not owned by any branch. Git only manages files that have been committed — untracked files are invisible to git's branch logic, so they physically sit in your working directory regardless of which branch you're on.

Real example: untracked `DPRD-3543` files followed checkout into `DPRD-3519` because git had no idea they existed.

Solution: `.git/info/exclude` — local-only ignore file, never committed, never pushed, invisible to teammates, permanent for your local copy.

This bootstrap auto-populates standard patterns. When you create *new* personal files, add them via the `gitignore-local-hygiene` skill — DO NOT add them to `.gitignore` (that affects the team).

12. Doc lookup routing (the source-of-truth rule)

Before claiming any external behavior, route to the MOST-SPECIFIC source:

Topic

Microsoft / Azure / .NET / VS Code / TypeScript on MS stack

Jira / Confluence / Atlassian

GitHub API / Actions / GraphQL

Browser / DOM / web platform

Any library / framework with public docs

Google Cloud, AWS, Stripe, Twilio, etc.

Postman

Codebase itself

Reaching for context7 for a Microsoft/Azure question is a smell — use microsoft-docs instead.

13. Parallelization protocol

Default: SERIALIZE. Parallelize only after the human approves. The parallelization-decision skill produces:

Stage A → Stage B

(none)

(data-only)

(files-overlap)

(unknown)

This applies to: implementation stages, parallel researchers in parallel-research-coordinator, multi-bug investigation, anything that fans out.

14. Plan folder organization

Every feature gets its own folder:

docs/ignored/implementation/<feature-slug>/
README.md (activity log)
Requirements.md
Plan-v1.md, Plan-v2.md, ..., Plan-Final.md
Review-1-*.md
Review-Consolidated-1-*.md
Parallelization-<ts>.md
Limitations.md
EdgeCases.md
Stages/Stage-1-*.md
Progress.md
PostMortem.md

Slug never contains the ticket ID, branch name, or any team-internal identifier. Good: oauth-login. Bad: dprd-3519.

15. Plan review cycle

implementation-planner → Plan-v1.md
↓ plan-review-cycle round 1
├─ requirements-alignment-reviewer
├─ architecture-enforcer
└─ feasibility-reviewer
↓ Review-Consolidated-1.md
↓ implementation-planner revises → Plan-v2.md
↓ round 2 (re-review)
↓ ... up to 5 rounds
↓ converged → Plan-Final.md (only implementable artifact)

If still issues after round 5 → escalate to human. Do not implement against a non-Final plan.

16. PR merge-conflict wait

After pr-creator opens a PR, pr-merge-conflict-wait:

1. Polls `gh pr view --json mergeable,mergeStateStatus,statusCheckRollup` every 5s up to 60s.
2. If **CONFLICTING** → STOP, report conflicts, hand off to merge-conflict-resolver only with human approval.
3. If **BEHIND** → recommend rebase.
4. If failing required checks → report.
5. Otherwise → PR ready.

PR is **never** "submitted" without this check.

17. Mobile / remote coding

Two ways to keep coding from your phone:

Option A: Claude.ai mobile app The official Claude mobile app (iOS / Android) lets you converse with Claude. On Max/Team plans the app exposes a Claude Code surface — you can send tasks to a remote Claude Code session and view results. The exact UX depends on your plan; check the in-app menu for "Claude Code" or "Coding mode." All your installed plugins (this one + the 15 official ones) work identically because they live on the machine running Claude Code, not on the phone.

Option B: SSH into a workstation that runs Claude Code Install a terminal app on your phone (Termius, Blink Shell, Tailscale SSH, etc.), SSH into a Mac/Linux workstation, and run Claude there. Your local repo, plugins, and `.git/info/exclude` all live on that workstation — the phone is just the keyboard. This works even when you can't get the mobile app's Claude Code surface.

Option C: Tailscale + Claude Code on a home Mac Same idea as B but Tailscale's mesh VPN gives you a stable hostname (`claudio.tail-xxxxx.ts.net`) you can SSH to from anywhere without port-forwarding.

In all three cases, the agents and skills run identically because they're loaded from the Claude Code install on the workstation. The phone is just an I/O device.

18. Update / uninstall

Update plugin:

```
/plugin marketplace update cc-orchestration  
/reload-plugins
```

Update bootstrap:

```
bash /path/to/install_repo_bootstrap.sh --repo /path/to/repo --yes # idempotent
```

Uninstall plugin:

```
/plugin uninstall claude-code-orchestration@cc-orchestration  
/plugin marketplace remove cc-orchestration
```

Undo bootstrap:

```
git config --unset core.hooksPath  
# manually clean docs/ignored/ + .git/info/exclude if desired  
rm -rf ~/.local/share/cc-orchestration
```

19. Troubleshooting

/plugin not recognized → upgrade Claude Code.

Plugin shows under Errors tab → open it. Common causes: hook script not executable, bad frontmatter, `${CLAUDE_PLUGIN_ROOT}` not expanded (upgrade Claude Code).

/agents missing specialists → orchestrator's tools: Agent(...) allowlist is the gate. To add an external agent, fork the plugin and extend the allowlist.

Git pre-commit blocking a legit commit → set `ALLOW_COMMENT_POLICY_BYPASS=1` (or `ALLOW_CLAUDE_ARTIFACT_COMMIT=1` for the file-pattern blocks) for that commit only.

Worktree didn't get docs/ignored → check `.worktreeinclude` in the source worktree; re-run `new-worktree.sh`.

Plugin install fails → ensure `/plugin marketplace add anthropics/claude-plugins-official` ran successfully first.

20. References

- Plugins (create): <https://code.claude.com/docs/en/plugins>
- Plugins (install): <https://code.claude.com/docs/en/discover-plugins>
- Plugin marketplaces: <https://code.claude.com/docs/en/plugin-marketplaces>
- Subagents: <https://code.claude.com/docs/en/sub-agents>
- Skills: <https://code.claude.com/docs/en/skills>
- Hooks: <https://code.claude.com/docs/en/hooks-guide>

- Worktrees: <https://code.claude.com/docs/en/worktrees>
- Agent teams: <https://code.claude.com/docs/en/agent-teams>
- Git ignore: <https://git-scm.com/docs/gitignore>
- Git hooks: <https://git-scm.com/docs/githooks>
- Git worktree: <https://git-scm.com/docs/git-worktree>
- claude.com/plugins (catalog browser)